

Household Command Center

Complete Product & Implementation Specification

For Cursor implementation — Web / PWA / iPad / Mobile / Tauri Desktop

Product principle: “Your household is under control.”

This is the complete 1–115 specification. It intentionally includes product vision, feature requirements, UX rules, CSS/theme tokens, data architecture, security, PWA, Tauri, AI, roadmap, and engineering acceptance criteria.

1. Product Vision

The application should answer one question immediately: “What do we need to know, remember, or do today?” It brings calendars, reminders, bills, subscriptions, documents, weather, contacts, places, important dates, and household information into one private workspace.

- Reduce mental overhead.
- Surface important information proactively.
- Make recurring household administration easy.
- Keep sensitive information private.
- Provide excellent tablet and desktop experiences.
- Work as a PWA.
- Run as a native desktop application through Tauri.
- Support two household members with shared and private information.
- Provide an extensible foundation for AI/agent capabilities later.

2. Product Principles

The product should behave like a calm household operating system rather than a generic CRUD dashboard.

- Calm, not overwhelming.
- Information density without visual clutter.
- Private by default.
- Fast to use.
- Excellent on iPad landscape.
- Excellent on desktop.
- Responsive on mobile.
- PWA-first.
- Tauri-compatible.
- AI-ready without making AI the core product.

3. Target Users

The initial application is designed for a household of two people. The data model should support more members later.

- Shared records are visible to both household members.
- Private records are visible only to the owner.
- Household membership is enforced at the backend level.

Shared task:
Pay electricity bill

Private task:
Complete personal certification

Shared document:
Rental agreement

Private document:
Personal passport

4. Target Platforms

Treat iPad landscape as the primary product experience, with desktop and mobile as first-class responsive experiences.

- iPad landscape target: 1194 × 834.
- Desktop: macOS, Windows, Linux.
- Mobile: iPhone and Android browsers plus installed PWA.

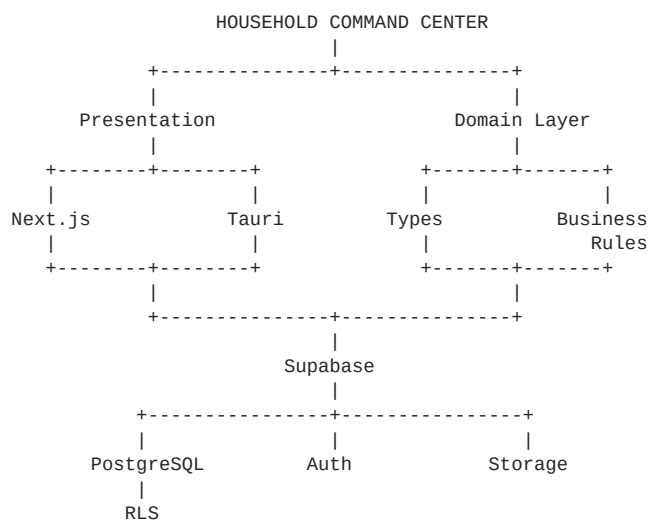
5. Technology Stack

Recommended implementation stack.

- Next.js + React + TypeScript
- Tailwind CSS + CSS variables/design tokens
- Supabase + PostgreSQL + Auth + Storage + RLS
- TanStack Query
- TanStack Table
- React Hook Form + Zod
- date-fns
- Lucide React
- Vercel
- Tauri
- pnpm

6. High-Level Architecture

Use a shared frontend/domain architecture so browser and Tauri clients do not duplicate business logic.



7. Monorepo Structure

Use pnpm workspaces and separate applications from reusable packages.

```
apps/
  web/
    Next.js application
  desktop/
    Tauri application

packages/
  ui/
  domain/
  types/
```

```

■■■ api/
■■■ config/

```

8. Application Navigation

Desktop/iPad uses a persistent left sidebar. Mobile uses a bottom navigation bar.

[illegible]

9. Visual Design Direction

The application should feel premium, modern, calm, professional, personal, trustworthy, elegant, and information-rich.

- Avoid enterprise ERP aesthetics.
- Avoid banking-portal aesthetics.
- Avoid crypto-dashboard aesthetics.
- Avoid generic Tailwind-template aesthetics.
- Use hierarchy, spacing, typography, and restrained color rather than decoration.

10. Brand Identity

Use Indigo as the primary brand identity with neutral surfaces.

- Indigo primary color.
- Warm neutral background.
- White surfaces.
- Subtle borders.
- Soft shadows.
- Inter typography.

11. Color System

Use CSS variables everywhere. Do not hard-code semantic colors inside feature components.

```
:root {
  --color-primary: #5046C7;
  --color-primary-hover: #4338B3;
  --color-primary-active: #392FA1;
  --color-primary-soft: #EDEBFF;

  --color-ink: #19171F;
  --color-muted: #63646F;
  --color-subtle: #8D8E98;
  --color-disabled: #B7B7BF;

  --color-background: #F6F6F8;
```

```

--color-surface: #FFFFFF;
--color-surface-subtle: #FAFAFC;
--color-surface-hover: #F7F7FA;

--color-border: #E4E4E9;
--color-border-strong: #D4D4DB;

--color-success: #168A54;
--color-success-soft: #E8F7EF;
--color-warning: #B9780B;
--color-warning-soft: #FFF5DD;
--color-danger: #C23A3F;
--color-danger-soft: #FDEBEC;
--color-info: #3971B8;
--color-info-soft: #EAF2FB;
}

```

12. Typography

Use Inter and maintain a restrained hierarchy.

- Page title: 28–32px / 700
- Section title: 18–20px / 600
- Card title: 15–17px / 600
- Body: 14–15px / 400
- Metadata: 12–13px / 500

```

:root {
  --font-sans: "Inter", system-ui, -apple-system,
    BlinkMacSystemFont, "Segoe UI", sans-serif;

  --text-xs: 0.75rem;
  --text-sm: 0.875rem;
  --text-md: 1rem;
  --text-lg: 1.125rem;
  --text-xl: 1.375rem;
  --text-2xl: 1.75rem;
  --text-3xl: 2.25rem;
}

```

13. Spacing System

Use an 8px-based spacing system.

```

:root {
  --space-1: 4px;
  --space-2: 8px;
  --space-3: 12px;
  --space-4: 16px;
  --space-5: 20px;
  --space-6: 24px;
  --space-8: 32px;
  --space-10: 40px;
  --space-12: 48px;
  --space-16: 64px;
}

```

14. Border Radius

Use soft but controlled rounding.

- Buttons: 8–10px
- Inputs: 8–10px
- Cards: 14–18px
- Large panels: 18–24px
- Badges: 999px

```

:root {
  --radius-sm: 6px;
}

```

```

--radius-md: 10px;
--radius-lg: 14px;
--radius-xl: 18px;
--radius-2xl: 24px;
--radius-pill: 999px;
}

```

15. Shadows

Prefer surface + border over heavy shadows.

```

:root {
  --shadow-card:
    0 1px 2px rgba(20, 20, 30, 0.04),
    0 8px 24px rgba(20, 20, 30, 0.04);

  --shadow-popover:
    0 12px 40px rgba(20, 20, 30, 0.12);

  --shadow-modal:
    0 24px 80px rgba(20, 20, 30, 0.18);
}

```

16. Motion

Motion should communicate state, not decorate everything.

- Sidebar selection: ~150ms
- Card hover: ~150ms
- Dropdown: ~180ms
- Dialog: ~200ms
- Page transitions: ~200–250ms
- Respect prefers-reduced-motion.

```

:root {
  --transition-fast: 120ms ease;
  --transition-normal: 180ms ease;
  --transition-slow: 280ms ease;
}

```

17. Accessibility

Target WCAG 2.2 AA.

- Keyboard navigation
- Visible focus states
- Semantic HTML
- Accessible dialogs
- Accessible forms
- Sufficient contrast
- Screen-reader labels
- Reduced-motion support

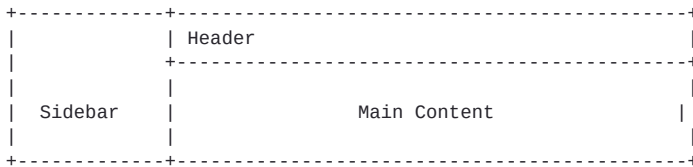
```

:focus-visible {
  outline: 2px solid var(--color-primary);
  outline-offset: 2px;
}

```

18. Application Shell

The shell contains sidebar, header, main content, command palette, notification center, and global quick-add action.



19. Dashboard — Purpose

The Dashboard is the most important screen. It should answer: What is happening today? What requires attention? What is coming next? What should I do next?

- Prioritize actionable information.
- Keep weather and secondary data visually subordinate.
- Use contextual microcopy.
- Aggregate information from other features.

20. Dashboard — Header and Attention

Show a contextual greeting and only surface attention when something genuinely needs action.

Good evening, Senthil.
 Everything important in one view.
 Tuesday, 6 October 2026

ATTENTION
 Residence Permit – Expires in 47 days
 Electricity – Payment due tomorrow

2 items need your attention

21. Dashboard — Weather

Show two compact weather contexts.

HOME
 14°
 Mostly cloudy
 Feels like 13°
 Antwerp

PARENTS
 29°
 Clear
 Dindigul

22. Dashboard — Today

Use a compact timeline for the day's schedule.

TODAY

09:30 Team Sync
 11:00 Dentist
 18:30 Dinner
 20:00 Read & Plan

23. Dashboard — Tasks

Show open tasks and a small progress summary.

TASKS
 5 open
 1 overdue

Complete
 ■■■■■■■■■■ 80%

Renew electricity contract

Prepare documents
Buy groceries

24. Dashboard — Payments

Surface the next few household payments.

UPCOMING PAYMENTS

Netflix	€17.99	10 Oct
Electricity	€82.40	Tomorrow
Internet	€49.00	15 Oct

25. Dashboard — Documents

Surface important document expiry information.

DOCUMENTS

Residence Permit
Expires in 47 days

Driving License
Valid until 2030

Passport
Valid until 2031

26. Dashboard — Layout

Recommended iPad landscape composition.

Sidebar	Header	
	Attention	
	Weather	Today
	Tasks	Payments
	Documents / Upcoming	

27. Calendar

Support Month, Week, and Agenda. Default to Agenda + mini calendar.

- Event fields: title, start time, end time, calendar, location, participants, notes.

OCTOBER 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
			1	2	3	4
5	6	7	8	9	10	11

Tuesday, 6 October
09:30 BNPPF Project Review
11:00 Dentist
18:30 Dinner

28. Tasks

Tasks support category, priority, owner, due date, visibility, recurring rules, and status.

- Categories: Personal, Family, Finance, Home, Work, Health, Travel, Other.
- Filters: All, Today, Upcoming, Overdue, Completed, Personal, Shared.
- Priority: Low, Medium, High, Urgent.

```
[ ] Renew electricity contract
    Finance · Shared
    Due tomorrow · High priority
```

29. Quick Add

Provide a global action for rapidly creating common records.

Add task
Add bill
Add subscription
Add document
Add important date
Add place
Add contact
Add account

Desktop shortcut:
Cmd/Ctrl + K

30. Bills

Bills are a primary household administration feature.

- Fields: name, provider, amount, currency, due date, frequency, category, owner, visibility, payment status, notes.

THIS MONTH
€1,248
Expected household expenses

€324
Upcoming

12
Active recurring items

31. Subscriptions

Track recurring services and upcoming renewals.

- Fields: name, provider, amount, currency, billing frequency, next renewal, category, owner, visibility, payment reference, notes, status.

Netflix
€17.99 / month
Next payment: 10 October
Shared

32. Renewal Intelligence

Show upcoming renewals in a compact list.

RENEWING SOON

Netflix	4 days
Domain	18 days
Insurance	42 days

33. Accounts

Accounts are reference information only; this is not a password manager.

- Fields: institution, account nickname, account type, reference, last 4 digits, owner, visibility, notes.
- Never store passwords, PINs, CVVs, OTPs, recovery codes, or authentication secrets.

ING
Household Current
.... 4832
Shared

34. Documents

Provide a lightweight household document vault.

- Primary documents: Passport, Residence Permit, Driving License, ID Card.

- Status: Valid, Expiring Soon, Expired, Missing.
- Initial file types: PDF, JPG, JPEG, PNG, WEBP, DOC, DOCX.
- Support drag-and-drop and native file selection.
- Use Supabase Storage with household-scoped policies.

IMPORTANT DOCUMENTS

Passport
Residence Permit
Driving License
ID Card

OTHER DOCUMENTS

Rental Agreement
Insurance Policy
Vehicle Document

35. Document Viewer

Opening a document should use a focused viewer with metadata and actions.

← Documents

Passport

```
+-----+
|               |
|       Document       |
|               |
+-----+
```

Owner: Senthil
Issued: ...
Expires: ...
Notes: ...

Download
Replace
Rename
Archive
Delete

36. Contacts

Provide a household-focused contact directory.

- Categories: Family, Friends, Doctors, Service Providers, Schools, Emergency, Other.
- Fields: name, phone, email, address, category, notes, owner, visibility.

Dr. John Smith
Dentist
+32 xxx xxx xxx
john@example.com

[Call] [Message] [Email]

37. Places

Places are personal saved locations for restaurants, weekend trips, family activities, hotels, travel, shopping, and places to revisit.

- Fields: name, address, website, rating, category, tags, notes, visited, saved by.

Ghent
Belgium
★★★★★
Weekend
Saved by both

38. Important Dates

A household memory system for birthdays, anniversaries, renewals, travel, family events, and deadlines.

- Categories: Birthdays, Anniversaries, Renewals, Travel, Family, Deadlines, Events, Other.

15 Oct – Father's Birthday
02 Nov – Insurance Renewal
20 Dec – Travel

39. Home

Manage practical household information.

- Maintenance
- Utilities
- Appliances
- Warranties
- Service Providers
- Home Inventory

Washing Machine
Purchased: June 2024
Warranty: Valid until June 2027

40. Home Maintenance

Track service schedules and household maintenance.

Boiler service
Due in 24 days

Air conditioner
Last serviced May 2026

Water filter
Replace in 12 days

41. Settings

Keep settings intentionally simple.

- Household identity
- Members
- Notifications
- Appearance
- Security
- Integrations
- Data

HOUSEHOLD
Household name
Members
Home location
Parents location
Timezone
Currency

NOTIFICATIONS
Bill reminders
Document expiry
Important dates
Task reminders
Subscription renewals
Daily briefing

QUIET HOURS
22:00 – 07:00

42. Global Search

Search should cover tasks, bills, subscriptions, documents, accounts, contacts, places, important dates, and home information.

Search: passport

Results:

Passport — Document

Passport renewal — Task

Passport expiry — Important Date

43. Command Palette

A premium command palette should make the application feel fast and powerful.

Search or type a command...

Add task

Add bill

Upload document

Open calendar

View expiring documents

View subscriptions

View important dates

Ask household assistant

44. Notifications

Notifications should be event-driven rather than noisy.

- Examples: document expiry, bill due tomorrow, subscription renewal, birthday tomorrow, overdue task.
- Priorities: Critical, High, Normal, Low.
- Respect quiet hours.

Residence permit expires in 30 days.

Electricity bill is due tomorrow.

Netflix renews tomorrow.

Dad's birthday is tomorrow.

Task is overdue.

45. Loading, Empty, and Error States

Every major feature needs deliberate loading, empty, and error states.

LOADING

Use skeletons that preserve final dimensions.

EMPTY

No upcoming payments

You're all caught up.

[Add payment]

ERROR

We couldn't load your subscriptions.

Check your connection and try again.

[Retry]

46. Responsive Design

Use platform-specific layouts rather than shrinking desktop UI.

- Desktop: persistent sidebar + multi-column content.
- iPad: persistent sidebar + two-column content.
- Mobile: top header + single-column content + bottom navigation.

```
:root {
```

```

--sidebar-width: 248px;
--sidebar-collapsed-width: 72px;
--content-max-width: 1600px;
}

```

47. Card Design Rules

Cards should have a clear header, content area, and optional action/footer.

UPCOMING PAYMENTS →	
Netflix	€17.99
10 October	
Electricity	€82.40
Tomorrow	

48. Component Architecture

Use a reusable UI system and feature-specific components.

```

components/
  ui/
    Button
    Input
    Select
    Checkbox
    Badge
    Card
    Dialog
    Dropdown
    Tabs
    Tooltip
    Avatar
    Skeleton
  layout/
    AppShell
    Sidebar
    Header
    PageContainer
    MobileNavigation
    CommandPalette
  dashboard/
  calendar/
  tasks/
  bills/
  subscriptions/
  documents/
  accounts/
  contacts/
  places/
  important-dates/
  home/
  settings/

```

49. Feature Architecture

Use feature-oriented architecture so business logic stays close to its domain.

```

features/
  dashboard/
  tasks/
  calendar/
  bills/
  subscriptions/
  documents/
  accounts/
  contacts/
  places/
  important-dates/
  home/

```

Each feature may contain:

```

components/
hooks/

```

```
schemas/  
types/  
queries/  
mutations/  
utils/
```

50. Forms

Use React Hook Form + Zod for validation and predictable form state.

```
Form  
↓  
React Hook Form  
↓  
Zod validation  
↓  
Mutation  
↓  
Supabase  
↓  
Query invalidation  
↓  
UI update
```

51. Data Fetching

Use TanStack Query for server state and caching.

```
useTasks()  
useUpcomingTasks()  
useBills()  
useUpcomingBills()  
useSubscriptions()  
useDocuments()  
useContacts()  
usePlaces()  
useImportantDates()
```

52. Database Model

Core tables should cover household membership, daily planning, administration, and personal information.

```
profiles  
households  
household_members  
  
tasks  
task_categories  
  
calendar_events  
  
bills  
subscriptions  
  
documents  
document_categories  
  
accounts  
contacts  
places  
  
important_dates  
  
home_items  
home_services  
  
notifications
```

53. Common Database Fields

Most tables should contain common audit and tenancy fields.

```
id  
household_id  
created_by  
created_at
```

updated_at

Optional:

owner_id

visibility

archived_at

54. Visibility Model

Applicable records should support shared/private visibility.

visibility = shared

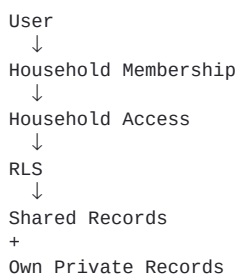
or

visibility = private

55. Supabase RLS

Security must be enforced at the database level.

- A user must belong to the household.
- Shared records are accessible to household members.
- Private records are accessible only to the owner.
- Never rely only on frontend filtering.



56. Storage

Store household documents in Supabase Storage using household-scoped paths and policies.

```
households/  
  {household_id}/  
    documents/  
      {document_id}/  
        file
```

57. Security UX

Communicate privacy without overwhelming the interface.

- Use subtle lock indicators for private information.
- Clearly distinguish shared and private records.
- Never expose sensitive fields unnecessarily.

■ Private
Only visible to you

58. Security Boundary

This application is not a password manager.

- Never store passwords.
- Never store PINs.
- Never store CVVs.

- Never store OTPs.
- Never store recovery codes.
- Never store authentication secrets.

59. Audit Metadata

For shared records, optionally show who created or updated the record.

Created by Senthil
Updated 2 days ago

60. Search and Filtering

All major list pages should provide search, filtering, sorting, and view controls.

Documents

Search documents...

All | Expiring | Expired | Shared | Personal

61. Sorting

Common sort options should be consistent across features.

- Recently updated
- Name
- Expiry date
- Due date
- Amount
- Owner

62. Iconography

Use one icon system consistently.

- Recommended: Lucide React.
- Typical sizes: 16px, 18px, 20px, 24px.
- Do not mix icon libraries.

63. Photography and Illustrations

Use imagery selectively because the application is primarily an information product.

- Good uses: Places, contacts, document previews, household memories.
- Avoid decorative stock imagery throughout the dashboard.

64. Theme Strategy

Initially ship Light mode, but design all tokens for future Dark and System themes.

- Light
- Dark
- System

```
[data-theme="dark"] {  
  --color-background: #111116;  
  --color-surface: #19191F;  
  --color-surface-subtle: #202027;
```

```
--color-border: #303039;  
--color-ink: #F5F5F7;  
--color-muted: #A5A5AF;  
--color-primary: #766CFF;  
--color-primary-hover: #8A82FF;  
--color-primary-soft: #29265A;  
}
```

65. Performance

Keep the app fast on iPad and desktop.

- Fast initial load.
- Fast navigation.
- Minimal layout shift.
- Optimistic mutations where appropriate.
- Lazy-load heavy views.
- Virtualize large tables.
- Avoid unnecessary client-side JavaScript.

66. Microcopy

Use contextual microcopy rather than generic CRUD labels.

Instead of:
Tasks

Prefer:
5 things to finish today

Instead of:
Bills

Prefer:
€324 coming up

Instead of:
Documents

Prefer:
2 documents need attention

67. Progressive Disclosure

Do not expose every property on every card.

Netflix
€17.99
Renews 10 Oct

Click:
Full subscription details

68. Motion Quality

Animations should be subtle and purposeful.

- Avoid excessive bounce or large transforms.
- Use 120–280ms transitions.
- Respect reduced-motion preferences.

69. Global UX Priority

Every screen should prioritize information in this order.

- 1. What requires attention?
- 2. What is happening today?
- 3. What is coming next?
- 4. What can I do quickly?
- 5. What can wait?

70. Mobile Navigation

Use a compact bottom navigation model.

Home
Calendar
Tasks
Documents
More

71. Desktop Sidebar

The desktop/iPad sidebar should be quiet and stable.

```
--sidebar-width: 248px  
--sidebar-collapsed-width: 72px
```

72. Content Width

Constrain large screens so information remains readable.

```
--content-max-width: 1600px
```

73. Table Behavior

Use TanStack Table for dense desktop/iPad data and transform tables into cards on mobile.

- Support sorting.
- Support filtering.
- Support row actions.
- Avoid horizontal scrolling where possible on mobile.

74. Forms UX

Forms should minimize cognitive load.

- Group related fields.
- Use clear labels.
- Validate near the field.
- Show errors in actionable language.
- Preserve user input after validation failures.

75. Delete and Archive

Destructive actions should be deliberate.

- Use confirmation for destructive deletion where recovery is not possible.
- Prefer archive for records that may be needed later.
- Use undo where practical.

76. Notification Center

Provide an in-app notification center in addition to push notifications.

- Unread count.
- Priority.
- Timestamp.
- Mark read.
- Deep-link to relevant record.

77. Notification Priority

Use four levels.

- Critical: expired document or deadline today.
- High: bill due tomorrow or document expiring soon.
- Normal: subscription renewal or upcoming event.
- Low: informational.

78. Daily Briefing

Optional daily briefing can summarize the household.

GOOD MORNING

Today:

3 calendar events

2 tasks due

1 payment tomorrow

No documents expiring today

Weather:

Home 14°

Parents 29°

79. Dashboard Personalization

Allow lightweight configuration without turning the dashboard into a widget builder.

- Show/hide secondary sections.
- Choose default calendar view.
- Choose default task filters.
- Keep core attention sections automatic.

80. Household Members

Support household members as first-class identities.

- Name
- Avatar
- Role
- Email
- Membership status
- Notification preferences

81. Authentication

Use Supabase Auth.

- Email/password may be supported.
- OAuth may be added later.
- Do not expose authentication secrets to the application database.

82. Household Onboarding

First-run onboarding should be short.

```
Create household
↓
Invite/add second member
↓
Set home location
↓
Set parents location
↓
Choose currency/timezone
↓
Open dashboard
```

83. Integrations

Keep integrations modular.

- Google Calendar
- Apple Calendar
- Weather provider
- Google Contacts
- Apple Contacts
- Email/notification provider

84. Calendar Integration

Calendar integration should normalize external events into a common presentation model.

- Provider
- External event ID
- Start/end
- Title
- Location
- Participants
- Sync status

85. Weather Integration

Weather should be provider-agnostic.

- Home location
- Parents location
- Current temperature
- Condition
- Feels-like
- Humidity

- Forecast

86. Contact Integration

Imported contacts should not silently overwrite manually curated household records.

- Use explicit sync/import.
- Preserve local notes.
- Track source/provider.
- Allow manual override.

87. Document Expiry Logic

Expiry dates should drive dashboard and notification behavior.

- 30-day warning.
- 7-day warning.
- Expired state.
- Configurable notification threshold later.

88. Bill Reminder Logic

Bills should produce reminders based on due date.

- Upcoming.
- Due tomorrow.
- Due today.
- Overdue.
- Paid/scheduled.

89. Subscription Logic

Subscriptions should surface renewal windows.

- Renewing soon.
- Renewing this month.
- Recently renewed.
- Cancelled/inactive.

90. Offline Strategy

Start with cached read access and the application shell.

- Do not attempt full offline replication in MVP.
- Use IndexedDB later for offline mutations.
- Design for sync conflicts rather than assuming they cannot occur.

91. PWA Requirements

The application should be installable and behave like an app.

- Web manifest.
- Standalone display.

- App icons.
- Splash/launch experience where supported.
- Service worker/offline shell.
- Push notifications where supported.

92. Tauri Strategy

Tauri should wrap the shared React application rather than become a second application.

- Reuse UI packages.
- Reuse domain logic.
- Reuse API abstractions.
- Keep native capabilities behind a small desktop adapter.

93. Tauri Desktop Features

Desktop-specific enhancements should improve convenience rather than duplicate functionality.

- System tray.
- Native notifications.
- Global shortcut.
- Quick Add.
- Native file picker.
- Open document.
- Launch at startup.
- Window management.

94. Tauri Server Boundary

Avoid bundling a full Next.js server into Tauri unless a strong requirement emerges.

- Prefer React frontend + Tauri shell + Supabase backend.
- Keep desktop lightweight.

95. AI Architecture

AI should be modular and optional.

- Household Assistant
- Personal Tech Radar
- Content Generator

96. Household Assistant

Allow natural-language questions over household data with explicit access boundaries.

What do we have coming up this week?
 Which bills need attention?
 What documents expire in the next 90 days?
 What subscriptions renew this month?
 What should we prepare for tomorrow?
 Plan our weekend based on weather and saved places.

97. Personal Tech Radar

A personal technology intelligence module can live inside the same application.

- React
- Next.js
- TypeScript
- Frontend architecture
- Web platform
- AI engineering
- Developer tools
- Leadership
- Career
- Mentoring

TODAY'S TECH RADAR

3 things worth knowing

1. React ecosystem update
2. New frontend architecture pattern
3. Developer tooling release

98. Content Generation Agent

Generate platform-specific content from one source idea.

- LinkedIn Post
- LinkedIn Article
- Medium Article
- Dev.to Article

```
Idea
↓
Research
↓
Outline
↓
Long-form article
↓
LinkedIn article
↓
LinkedIn post
↓
Medium / Dev.to version
```

99. AI Data Access

AI should access only the data needed for the current task.

- Use explicit tool/domain boundaries.
- Do not expose secrets.
- Log important AI actions where appropriate.
- Keep AI optional.

100. MVP Phase 1 — Foundation

Build the foundation before feature breadth.

- Authentication
- Household

- Household members
- RLS
- Design system
- Application shell
- Responsive navigation

101. MVP Phase 2 — Core Daily Experience

Build the daily workflow.

- Dashboard
- Tasks
- Calendar
- Important Dates

102. MVP Phase 3 — Household Administration

Build the highest-value administrative features.

- Bills
- Subscriptions
- Documents
- Accounts

103. MVP Phase 4 — Personal Information

Add household reference features.

- Contacts
- Places
- Home

104. MVP Phase 5 — PWA

Make the product installable and useful outside a browser tab.

- Installability
- Offline shell
- Caching
- Notifications
- Push notifications

105. MVP Phase 6 — Desktop

Add Tauri after the web/PWA core is stable.

- Tauri
- System tray
- Native notifications
- Global shortcut
- Quick Add

- Native file picker

106. MVP Phase 7 — AI

Add intelligence after the core product is reliable.

- Household Assistant
- Personal Tech Radar
- Content Generator

107. Definition of Done

A feature is not complete when only its happy-path UI works.

- UI
- Responsive layout
- Loading state
- Empty state
- Error state
- Validation
- Data persistence
- RLS/security
- Mobile behavior
- Accessibility
- Keyboard behavior
- Success feedback

108. Testing Strategy

Each feature should have unit, component, and end-to-end coverage proportional to risk.

- Unit tests for business rules.
- Component tests for complex UI states.
- E2E tests for critical flows: login, task creation, payment creation, document upload, permissions.

109. Quality Gates

Before merging a feature, verify functionality and UX.

- TypeScript passes.
- Lint passes.
- Tests pass.
- Production build passes.
- Responsive layouts checked.
- Accessibility checked.
- RLS behavior checked.
- No secrets committed.

110. Visual QA

Every screen should be visually reviewed at iPad landscape and desktop widths.

- Check alignment.
- Check whitespace.
- Check typography.
- Check truncation.
- Check loading/empty/error states.
- Check dark-theme token readiness.
- Check mobile transformation.

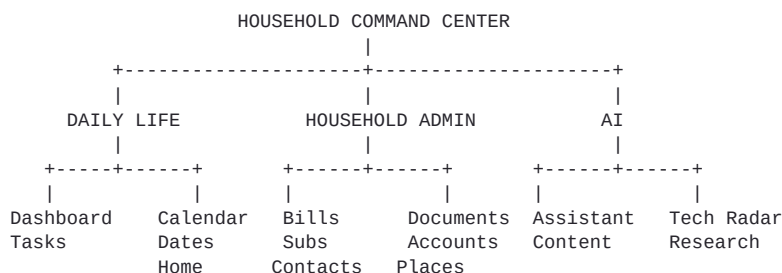
111. Security QA

Security is part of feature completion.

- Verify RLS policies.
- Verify private records cannot be read by the other household member.
- Verify Storage policies.
- Verify server-side authorization.
- Verify no sensitive credentials are stored.

112. Final Product Structure

The product is a household operating system composed of daily life, administration, and optional AI.



113. Core Product Experience

The ideal user flow should be fast and decisive.

```
graph TD
    A[Open app] --> B[See dashboard]
    B --> C[Understand today's situation]
    C --> D[Notice anything requiring attention]
    D --> E[Complete quick actions]
    E --> F[Review upcoming events/payments]
    F --> G[Close app]
```

114. Product Personality

The final product should communicate calm, clarity, confidence, preparedness, and trust.

- Avoid productivity pressure.
- Avoid administrative complexity.
- Avoid information overload.

115. Implementation Priority and Final Direction

Implement the system in a deliberate order and treat the Dashboard as the contextual layer over all feature domains.

- 1. Design tokens
- 2. UI component system
- 3. App shell
- 4. Authentication
- 5. Household model
- 6. Dashboard
- 7. Tasks
- 8. Calendar
- 9. Bills
- 10. Subscriptions
- 11. Documents
- 12. Important Dates
- 13. Accounts
- 14. Contacts
- 15. Places
- 16. Home
- 17. Notifications
- 18. Search
- 19. Command Palette
- 20. PWA
- 21. Tauri
- 22. AI Assistant
- 23. Tech Radar
- 24. Content Generator

```
Tasks      \  
Bills      \  
Calendar  ---> Dashboard  
Documents  /  
Dates      /
```

Individual feature pages manage data.
The Dashboard provides context.

Final technical direction:
Next.js + React + TypeScript
Tailwind + CSS variables
React Hook Form + Zod
TanStack Query + TanStack Table
Supabase + PostgreSQL + Auth + Storage + RLS
Vercel
Tauri
Lucide React
pnpm
Separate AI service/domain layer

Implementation Note for Cursor

Use this document as the product source of truth. Implement feature-by-feature in the stated priority order. Preserve the design token system, shared domain model, RLS boundaries, responsive behavior, and acceptance criteria. Do not reduce the product to a collection of generic CRUD screens.